

透過 Private Service Connect 端點，使用 Python SDK 存取 Gemini 3 Pro 對話

來源：<https://codelabs.developers.google.com/codelabs/terraform-python-vertexai-psc?hl=zh-tw>

步驟數：10

透過 Python SDK 和 PSC 端點，從 VM 存取 Gemini 3 Pro

目錄

1. [1. 總覽](#)
2. [2. 目標](#)
3. [3. 工作 1：使用 Terraform 設定環境](#)
4. [4. 工作 2：使用 Terraform 建立 NAT 閘道和 VM](#)
5. [5. 工作 3：設定 VM 並進行測試](#)
6. [6. 工作 4：使用 Terraform 建立 googleapis 的 PSC 端點](#)
7. [7. 工作 5：使用 Terraform 建立 googleapis 的手動 DNS 項目](#)
8. [8. 工作 6：透過 IP 位址驗證端點連線](#)
9. [9. 清除](#)
10. [10. 恭喜](#)

1. 總覽

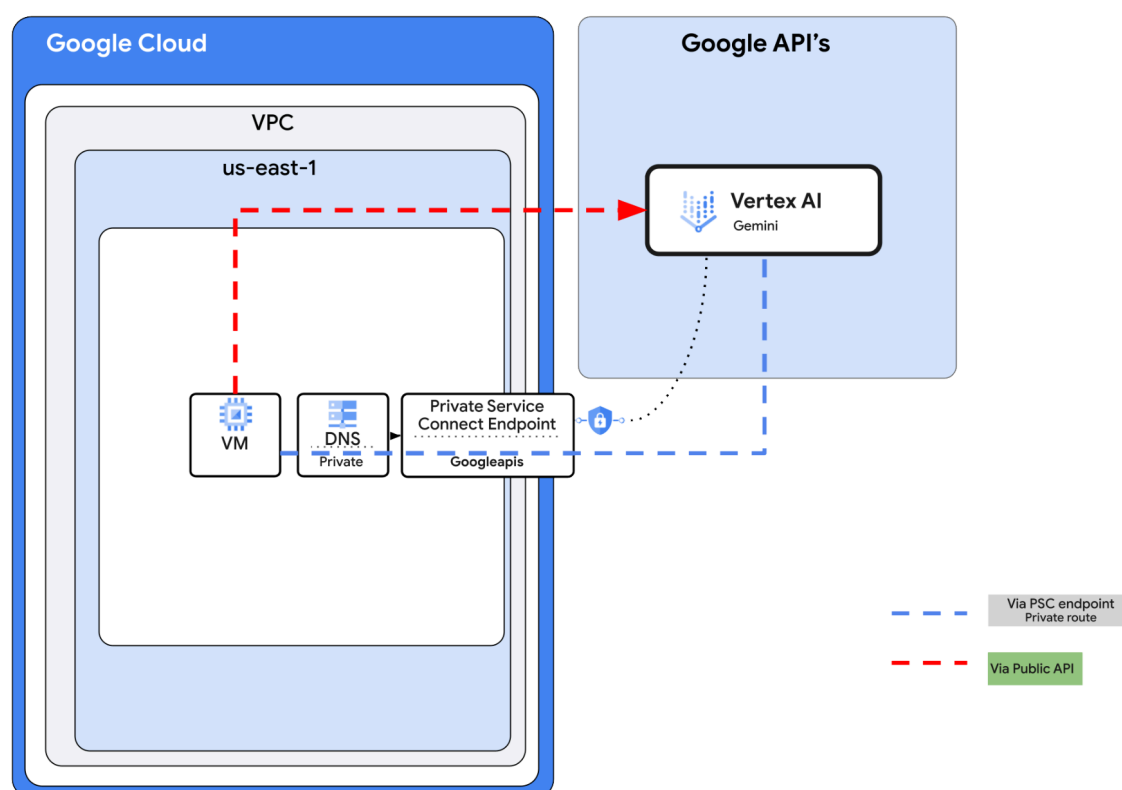
Vertex AI API 可透過網際網路存取，但您可能希望在企業中私下存取 Vertex AI API，而不透過網際網路。在本實驗室中，您將先執行下列操作：

- 透過 Python SDK，使用 Vertex 存取 **Gemini 3 Pro API**
- 這項作業會在 VM 執行個體上執行
- 連線會透過 Cloud NAT 連線至公開網際網路。

接著，您將建立 Google API 的 Private Service Connect 端點，並變更流量流程，使用私人端點連線至 Gemini 對話 API。設定會結合 Terraform、gcloud 和控制台。

在本實驗室中，您將建立下列模式。

圖 1。



2. 目標

在本實驗室中，您將瞭解如何執行下列工作：

- 設定 VM 執行個體以使用 Python SDK
- 透過 Python 指令碼連線至 Gemini 對話
- 設定 PSC 端點，連線至 [Googleapis](#)
- 驗證 Googleapis 的連線路徑
- 設定手動 DNS 項目

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar for projects and folders. A blue arrow points from the 'Select a project' dropdown to the search bar. To the right of the search bar is a 'NEW PROJECT' button with a gear icon. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the value 'My Project 13475', a 'Project ID' field with the value 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. The 'Project ID' field has a note that it cannot be changed later and an 'EDIT' link. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell terminal. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal has a dark background and displays the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

3. 工作 1：使用 Terraform 設定環境

我們會建立含有防火牆規則和子網路的自訂虛擬私有雲。開啟 **Cloud 控制台**，然後選取要使用的專案。

1. 開啟控制台右上方的 Cloud Shell，確認 Cloud Shell 中顯示的專案 ID 正確無誤，並確認允許存取的任何提示。

Search (/) for resources, docs, products and more

Search



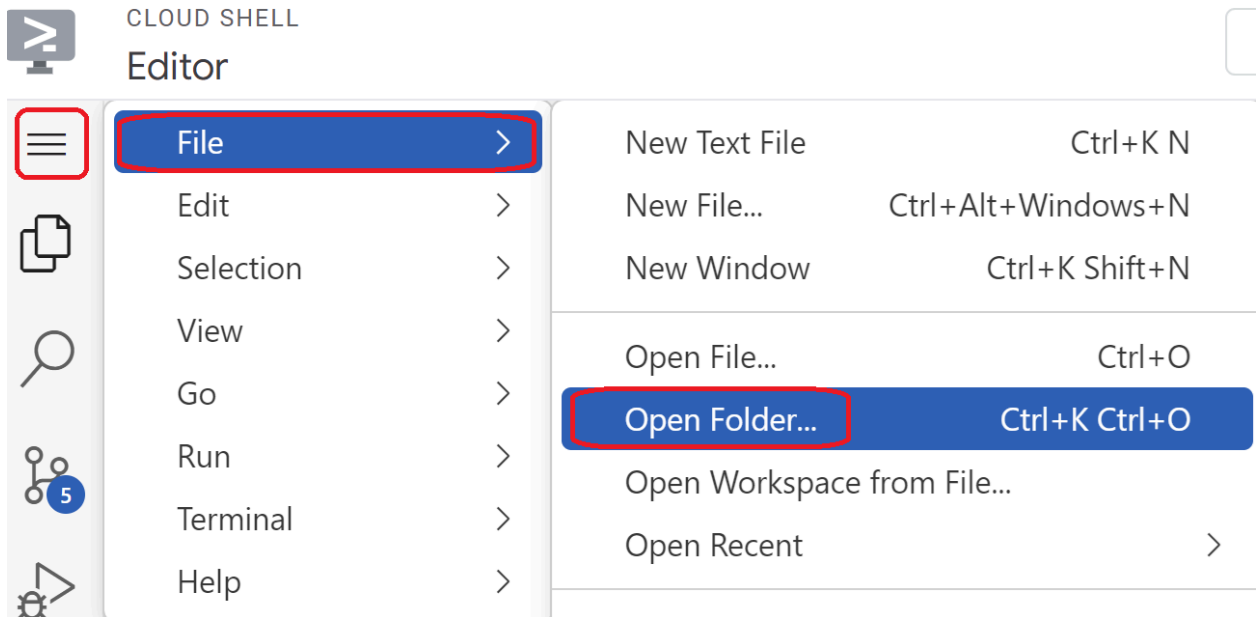
2. 建立名為 terraform-build 的資料夾，然後移至該資料夾

```
mkdir terraform-build && cd terraform-build
```

3. 建立 **main.tf** 和 **variable.tf** 檔案。

```
touch main.tf variable.tf
```

4. 切換至 **Cloud Shell 編輯器**檢視畫面。選取「編輯器」，並允許所有必要提示，確保介面可以載入。
5. 載入後，依序前往「File」>「Open Folder」，然後前往「/home/your-user-name/terraform-build」，並選取「Ok」，在編輯器中開啟資料夾。



6. 選取 **variable.tf** 檔案，然後新增下列內容。將 **your-project-id-here** 文字替換為實際專案 ID (以半形引號括住)

```
variable "project_id" {
  type = string
  default = "your-project-id-here"
}

variable "network_id" {
  type = string
  default = "python-net"
}
```

7. 接著開啟 **main.tf** 檔案。我們將新增一些 Terraform 程式碼，執行下列各種動作。

啟用 API	<code>resource "google_project_service" "default"</code>
建立名為 <i>python-net</i> 的虛擬私有雲	<code>resource "google_compute_network" "default"</code>
新增子網路	<code>resource "google_compute_subnetwork" "default"</code>
新增兩項防火牆規則	<code>resource "google_compute_firewall" "allow_icmp"</code> <code>resource "google_compute_firewall" "allow_ssh"</code>

8. 複製下列程式碼並貼到 **main.tf** 檔案中。

```

resource "google_project_service" "default" {
  project = var.project_id
  for_each = toset([
    "dns.googleapis.com",
    "aiplatform.googleapis.com",
    "servicedirectory.googleapis.com"
  ])

  service          = each.value
  disable_on_destroy = false
}

resource "google_compute_network" "default" {
  project      = var.project_id
  name         = var.network_id
  auto_create_subnetworks = false
  mtu          = 1460
  routing_mode = "GLOBAL"
}

resource "google_compute_subnetwork" "default" {
  name           = "vm1-subnet"
  ip_cidr_range = "10.0.11.0/24"
  project        = var.project_id
  region         = "us-east1"
  stack_type     = "IPV4_ONLY"
  network        = google_compute_network.default.id
}

resource "google_compute_firewall" "allow_icmp" {
  name      = "allow-icmp-${google_compute_network.default.name}"
  network   = google_compute_network.default.id
  project   = var.project_id

  allow {
    protocol = "icmp"
  }

  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["allow-icmp"]
}

resource "google_compute_firewall" "allow_ssh" {
  name      = "allow-ssh-${google_compute_network.default.name}"
  network   = google_compute_network.default.id
  project   = var.project_id

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
}

```



```
source_ranges = ["0.0.0.0/0"]
target_tags   = ["allow-ssh"]
}
```

9. 切換回 **Cloud Shell** 終端機，確認您位於 **terraform-build** 目錄 `cd terraform-build`，然後執行下列指令：

```
terraform init
```

初始化工作目錄。這個步驟會下載指定設定所需的供應商。

```
terraform plan
```

產生執行計畫，顯示 Terraform 將採取哪些動作來部署基礎架構。

10. 現在請執行 `terraform apply` 指令，然後輸入 `yes` 來建立資源。

驗證設定：檢查環境是否符合下列條件：

- 1 個名為 python-net 的虛擬私有雲
- 1 個子網路
- 2 項防火牆規則

步驟 4 · 約 0 分鐘

4. 工作 2：使用 Terraform 建立 NAT 閘道和 VM

我們需要授予連出網際網路的外部存取權，因此請建立並附加 Cloud NAT 閘道。

1. 開啟 Cloud Shell，前往 terraform-build 資料夾，然後建立下列檔案 (共三個檔案)。我們稍後會編輯這些內容。

```
touch nat-vm.tf psc.tf dns.tf
```

2. 切換至 **Cloud Shell 編輯器**檢視畫面，選取 **nat-vm.tf** 檔案，然後新增下列 Terraform 程式碼。系統會建立 NAT 閘道和兩部 VM。

Terraform ***nat-vm.tf***

```

resource "google_compute_router" "default" {
  name      = "py-outbound-nat"
  region    = "us-east1"
  network   = google_compute_network.default.id
  project   = var.project_id

  bgp {
    asn = 64514
  }
}

resource "google_compute_router_nat" "default" {
  name                        = "py-outbound-nat-gw"
  router                     = google_compute_router.default.name
  region                     = google_compute_router.default.region
  nat_ip_allocate_option     = "AUTO_ONLY"
  project                    = var.project_id
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"

  log_config {
    enable = true
    filter = "ERRORS_ONLY"
  }
}

resource "google_compute_instance" "vm1" {
  name          = "py-vm1"
  zone          = "us-east1-b"
  machine_type  = "n2-standard-2"
  project       = var.project_id

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }

  network_interface {
    subnetwork = google_compute_subnetwork.default.id
    stack_type = "IPV4_ONLY"
  }

  tags = ["allow-ssh", "allow-icmp"]

  metadata_startup_script = <<-EOF
    #! /bin/bash
    sudo apt-get update
    sudo apt-get install python3 python3-dev python3-venv git -y
    sudo apt-get install tcpdump dnsutils -y

    mkdir -p ~/py-gem-env
  >>>EOF

```

```
cd ~/py-gem-env
python3 -m venv env
source env/bin/activate
pip install --upgrade pip
pip install ipython google-genai
EOF
}
```

3. 切換至 **Cloud Shell** 終端機，確認您位於 **terraform-build** 資料夾中，然後執行 `terraform plan`，這會顯示將新增 4 個項目，接著執行 `terraform apply` 並輸入 `yes`，建立網路位址轉譯 (NAT) 閘道和 VM。

驗證設定：檢查環境是否符合下列條件：

1 個 NAT 閘道

1 個 Cloud Router

名為「`py-vm1`」的 VM

5. 工作 3：設定 VM 並進行測試

1. 前往 VM 執行個體。選取以 **py-vm1** 開頭的 VM。選擇「SSH」**SSH**。
2. 透過 SSH 連線至 **py-vm1** 後，輸入 **sudo -i** 啟用根權限。
3. 啟動 **venv** 環境：

```
cd py-gem-env  
source env/bin/activate
```

4. 現在請驗證這項設定，以便稍後進行測試。在 VM 中執行下列指令，並在系統提示時按下 **y**。

```
gcloud auth application-default login
```

5. 接著複製開頭為 **https://** 的網址，在實驗室瀏覽器視窗中開啟新分頁並貼上網址。接受提示。
6. 看到下列訊息時，請選取複製，然後切換回 VM **py-vm1** 工作階段，在 **Enter authorization code:** 中貼上複製的授權碼，然後按下 **Enter** 鍵進行驗證。



Sign in to the gcloud CLI

You are seeing this page because you ran the following command in the gcloud CLI from this or another machine. If this is not the case, close this tab.

```
gcloud auth application-default login  
--no-launch-browser
```

Enter the following authorization code in gcloud CLI on the machine you want to log into. This is a credential **similar to your password** and should not be shared with others.

Copy

You can close this tab when you're done.

7. 現在讓我們快速測試是否可以連線至 Vertex API，這會使用 ***-aiplatform.googleapis.com**，因此我們會對該位址執行 **dig**，瞭解流量的路由方式。

```
dig *-aiplatform.googleapis.com
```

8. 畫面應會顯示類似內容 (地址會有所不同)。注意：由於 API 是公用 API，路徑會經過公開 IP 位址。請勿複製

```
; <<>> DiG 9.18.41-1-deb12u1-Debian <<>> *-aiplatform.googleapis.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 60947
;; flags: qr rd ra; QUERY: 1, ANSWER: 16, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
*-aiplatform.googleapis.com.      IN      A

;; ANSWER SECTION:
*-aiplatform.googleapis.com. 300 IN      A      173.194.216.95
*-aiplatform.googleapis.com. 300 IN      A      108.177.11.95
*-aiplatform.googleapis.com. 300 IN      A      192.178.219.95
*-aiplatform.googleapis.com. 300 IN      A      74.125.134.95
*-aiplatform.googleapis.com. 300 IN      A      74.125.139.95
*-aiplatform.googleapis.com. 300 IN      A      108.177.12.95
*-aiplatform.googleapis.com. 300 IN      A      173.194.210.95
*-aiplatform.googleapis.com. 300 IN      A      74.125.26.95
*-aiplatform.googleapis.com. 300 IN      A      173.194.212.95
*-aiplatform.googleapis.com. 300 IN      A      172.217.204.95
```

9. 現在我們來使用 Python。輸入 `ipython` 即可啟用 `ipython` 介面。

```
ipython
```

```
(env) root@py-vm1-east:~/py-gem-env# ipython
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.18.1 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

10. 複製並貼上下列內容。這會要求 Gemini 「Google 標誌的所有顏色」和「天空的顏色」。

```

from google import genai
from google.genai import types
import os
import sys

LOCATION = "global"
MODEL_ID = "gemini-3-pro-preview"

try:
    client = genai.Client(vertexai=True, location=LOCATION)
    print(f"Successfully initialized Google Gen AI Client (Vertex AI mode) in {LOCATION}")
except Exception as e:
    print(f"Error initializing client: {e}")
    print("Ensure you have installed the library: `pip install google-genai`")
    print("And authenticated: `gcloud auth application-default login`")
    sys.exit(1)

class SimpleChatSession:
    def __init__(self, model_id):
        self.model_id = model_id
        self.history = []

    def send_message(self, prompt, stream=True):
        user_content = types.Content(
            role="user",
            parts=[types.Part(text=prompt)]
        )
        self.history.append(user_content)

        try:
            response_stream = client.models.generate_content_stream(
                model=self.model_id,
                contents=self.history,
                config=types.GenerateContentConfig(
                    temperature=0.7
                )
            )

            accumulated_text = ""

            for chunk in response_stream:
                if chunk.text:
                    yield chunk.text
                    accumulated_text += chunk.text

            model_content = types.Content(
                role="model",
                parts=[types.Part(text=accumulated_text)]
            )
            self.history.append(model_content)

        except Exception as e:

```

```

        print(f"\n[Error during generation: {e}]\n")

def get_chat_response(session: SimpleChatSession, prompt: str):
    print(f"\n--- User: {prompt} ---")
    print("--- Model: ", end="")

    stream_generator = session.send_message(prompt)

    full_text = ""
    for chunk_text in stream_generator:
        full_text += chunk_text
        print(chunk_text, end="", flush=True)

    print()
    return full_text

if __name__ == "__main__":
    chat_session = SimpleChatSession(MODEL_ID)

    get_chat_response(chat_session, "Hello.")
    get_chat_response(chat_session, "What are all the colors of the Google logo?")
    get_chat_response(chat_session, "What color is the sky?")

```

11. 按兩次 Enter 鍵即可執行並查看結果。
12. 這項要求透過公用 API 存取 Vertex。
13. 關閉 **SSH** 工作階段，然後繼續。

注意：你可以將提示問題修改為其他內容。

6. 工作 4：使用 Terraform 建立 googleapis 的 PSC 端點

如要啟用與 Vertex API 端點的私人連線，請為 Google API 建立 Private Service Connect 端點。這樣一來，我們就能使用指派的私人 IP 位址，將流量轉送至所需的 Google API (在本例中為 Vertex)。

1. 如果尚未開啟 Cloud Shell，請在編輯器檢視畫面中開啟。我們要建立下列項目：

- 為 PSC 端點 **192.168.255.250** 建立 IP (`resource "google_compute_global_address" "default"`)
- 建立 Google API 的 PSC 端點 (`resource "google_compute_global_forwarding_rule" "default"`)

開啟 **terraform-build** 資料夾中的 **psc.tf** 檔案。在檔案中新增下列程式碼。

Terraform **psc.tf**

```
resource "google_compute_global_address" "default" {
  name          = "vertex-ip"
  purpose       = "PRIVATE_SERVICE_CONNECT"
  network       = google_compute_network.default.id
  address_type  = "INTERNAL"
  address       = "192.168.255.250"
}

resource "google_compute_global_forwarding_rule" "default" {
  name          = "pscvertexgemini"
  target        = "all-apis"
  network       = google_compute_network.default.id
  ip_address    = google_compute_global_address.default.id
  load_balancing_scheme = ""
}
```

2. 切換到 **Cloud Shell** 終端機，確認您位於 `terraform-build` 資料夾。然後執行 `terraform init` 然後執行 `terraform plan`，這會顯示將新增 2 個項目，然後執行 `terraform apply` 並輸入 `yes`，即可建立 IP 和 PSC Google API 端點。
3. 確認端點是否存在

```
gcloud compute addresses list --filter="name=( 'vertex-ip' ...)"
```

```
gcloud compute forwarding-rules describe pscvertexgemini --global
```

7. 工作 5：使用 Terraform 建立 googleapis 的手動 DNS 項目

您可以使用私人 DNS 建立手動 DNS 項目，指向 PSC 端點。這會影響您指派給該廣告活動的所有聯播網。

1. 前往「網路服務」，然後選取「Cloud DNS」。
2. 在區域中，您應該會看到系統自動為 Google API 適用的 Private Service Connect 建立區域，且區域類型為 Service Directory。可用於以 *****SERVICE-ENDPOINT.p.googleapis.com*** 格式連線至 PSC 端點。範例：`aiplatform-pscvertexgemini.p.googleapis.com`
3. 在本例中，我們要手動建立私人 DNS 項目。設定如下所示
 - 為「googleapis.com」建立名為「googleapis-private」的私人 DNS 區域，並將其限制在「python-net」網路中。
 - 新增 A 記錄，將「googleapis.com」對應至 IP 位址「192.168.255.250」。
 - 新增 CNAME 記錄，將「googleapis.com」的所有子網域 (例如 `www.googleapis.com`) 重新導向至「googleapis.com」。
4. 如果尚未開啟 Cloud Shell，請在編輯器檢視畫面中開啟。開啟 terraform-build 資料夾中的 **dns.tf** 檔案。在檔案中新增下列程式碼。

Terraform ***dns.tf***

```

resource "google_dns_managed_zone" "private_zone" {
  name      = "googleapis-private"
  dns_name  = "googleapis.com."
  visibility = "private"
  project   = var.project_id

  private_visibility_config {
    networks {
      network_url = google_compute_network.default.id
    }
  }
}

resource "google_dns_record_set" "a_record" {
  name      = "googleapis.com."
  type      = "A"
  ttl       = 300
  managed_zone = google_dns_managed_zone.private_zone.name
  project    = var.project_id

  rrdatas = ["192.168.255.250"]
}

resource "google_dns_record_set" "cname_record" {
  name      = "*.googleapis.com."
  type      = "CNAME"
  ttl       = 300
  managed_zone = google_dns_managed_zone.private_zone.name
  project    = var.project_id

  rrdatas = ["googleapis.com."]
}

```

5. 切換到 **Cloud Shell** 終端機，確認您位於 `terraform-build` 資料夾。然後執行 `terraform plan`，這會顯示要新增的項目，

接著執行 `terraform apply` 並輸入 `yes`，建立私人 DNS 項目。

6. 您應該會看到設定，其中包含 **A** 記錄和 **CNAME**，如下所示：

googleapis-private

DNS name	googleapis.com.
Type	Private

RECORD SETS

IN USE BY

[+ ADD STANDARD](#)

[+ ADD WITH ROUTING POLICY](#)

[🗑 DELETE RECORD SETS](#)

[🔄 REFRESH](#)

 Filter Filter record sets





☐	DNS name 	Type	TTL (seconds)	Routing policy		
☐	*.googleapis.com.	CNAME	300	Default		
☐	googleapis.com.	SOA	21600	Default		
☐	googleapis.com.	NS	21600	Default		
☐	googleapis.com.	A	300	Default		

7. 接下來，我們要在 **py-vm1** 上驗證這些變更的連線能力

8. 工作 6：透過 IP 位址驗證端點連線

讓我們使用私人端點連線至 Gemini。

注意：您不必在 VM 上手動設定。

1. 前往 VM 執行個體 **py-vm1**。選取「SSH」，透過 SSH 連線至 VM
2. 輸入 **sudo -i** 取得根存取權
3. 使用 **ping** 指令檢查 **aiplatform.googleapis.com** 的連線路徑。這會對 googleapis 的私人 DNS 中的 IP 位址 (A 記錄) 執行 Ping 作業。這個 IP 是 PSC 端點，因此 Ping 會失敗。

```
ping -c 2 aiplatform.googleapis.com
```

4. 使用 **aiplatform-pscvertexgeminip.googleapis.com** 為 PSC Google API 自動建立的 DNS 項目，以 **ping** 檢查連線路徑。這會指向 PSC 端點的 IP 位址，因此您無法成功執行 Ping。

```
ping -c 2 aiplatform-pscvertexgeminip.googleapis.com
```

5. 使用 **dig** 指令檢查 **aiplatform.googleapis.com** 的連線路徑。這應該是 PSC 端點的 IP 位址。

```
dig aiplatform.googleapis.com
```

6. 返回控制台，開啟另一個 VM 執行個體 **py-vm1**。選取「SSH」**SSH**並透過 SSH 連線至 VM
7. 輸入 **sudo -i** 取得根存取權
8. 執行下列指令，即可在 TCP 傾印中查看連線能力

```
sudo tcpdump -i any port 53 -n or host aiplatform.googleapis.com
```

9. 現在切換回 VM 執行個體 **py-vm1** 的第一個 SSH 執行個體
10. 使用下列指令啟用環境：

```
cd py-gem-env  
source env/bin/activate
```

11. 現在來測試 Python。輸入 **ipython** 即可啟用 **ipython** 介面。

```
ipython
```

12. 複製並貼上下列內容。這會要求 Gemini「用一句簡短的話，摘要說明 AI 領域中的權杖化工具是什麼？」和「鯨魚很棒，對吧？」。

```

from google import genai
from google.genai import types
import os
import sys

LOCATION = "global"
MODEL_ID = "gemini-3-pro-preview"

try:
    client = genai.Client(vertexai=True, location=LOCATION)
    print(f"Successfully initialized Google Gen AI Client (Vertex AI mode) in {LOCATION}")
except Exception as e:
    print(f"Error initializing client: {e}")
    print("Ensure you have installed the library: `pip install google-genai`")
    print("And authenticated: `gcloud auth application-default login`")
    sys.exit(1)

class SimpleChatSession:
    def __init__(self, model_id):
        self.model_id = model_id
        self.history = []

    def send_message(self, prompt, stream=True):
        user_content = types.Content(
            role="user",
            parts=[types.Part(text=prompt)]
        )
        self.history.append(user_content)

        try:
            response_stream = client.models.generate_content_stream(
                model=self.model_id,
                contents=self.history,
                config=types.GenerateContentConfig(
                    temperature=0.7
                )
            )

            accumulated_text = ""

            for chunk in response_stream:
                if chunk.text:
                    yield chunk.text
                    accumulated_text += chunk.text

            model_content = types.Content(
                role="model",
                parts=[types.Part(text=accumulated_text)]
            )
            self.history.append(model_content)

        except Exception as e:

```

```

        print(f"\n[Error during generation: {e}]")

def get_chat_response(session: SimpleChatSession, prompt: str):
    print(f"\n--- User: {prompt} ---")
    print("--- Model: ", end="")

    stream_generator = session.send_message(prompt)

    full_text = ""
    for chunk_text in stream_generator:
        full_text += chunk_text
        print(chunk_text, end="", flush=True)

    print()
    return full_text

if __name__ == "__main__":
    chat_session = SimpleChatSession(MODEL_ID)

    get_chat_response(chat_session, "Hello.")
    get_chat_response(chat_session, "In one short sentence summarize what is a tokenizer in the context of AI?")
    get_chat_response(chat_session, "Are whales awesome or not?")

```

13. 按兩次 **Enter** 鍵即可執行並查看結果。

14. 切換回 VM 執行個體 **py-vm1** 的第二個執行個體。您應該會看到 TCPDUMP 的結果。您會發現 VM 的輸入和輸出，以及 IP 位址，都使用 PSC 端點 IP 位址連線至 **aiplatform.googleapis.com**。

關閉 VM 執行個體 **py-vm1** 的所有 SSH 工作階段

步驟 9 · 約 0 分鐘

9. 清除

1. 前往 **Cloud Shell**，確認您位於 **terraform-build** 目錄 `cd terraform-build`
2. 執行 `terraform plan destroy`，查看所有將進行的變更

```
terraform plan -destroy
```

3. 接著執行 `terraform destroy` 指令，然後輸入 `yes`，即可移除您在專案中透過 Terraform 建立的所有資源。

```
terraform destroy
```

步驟 10 · 約 0 分鐘

10. 恭喜

恭喜！您已成功連線至 Vertex，並透過公用 API 位址使用 **Gemini 3 Pro**，以及透過 Google API 適用的 Private Service Connect 端點私下使用。這項功能可將非公開 API 連線能力擴展至地端部署/其他雲端環境，這些環境透過 (互連網路、Cross-Cloud Interconnect 和虛擬私有雲) 連線。

後續步驟/瞭解詳情

如要進一步瞭解 Vertex AI 網路，請參閱[這篇文章](#)。

程式碼研究室：[透過 Private Service Connect 端點，使用 Python SDK 在 Vertex AI 存取 Anthropic Claude](#)

挑戰下一個實驗室

繼續完成 Google Cloud 任務，或查看其他 Google Cloud Skills Boost 實驗室：

- [Google Cloud 網路基礎知識](#)
 - [探索生成式 AI - Vertex AI](#)
 - [Google Cloud 上的代理式 AI](#)
-